

Plateforme de transfert sécurisé de fichiers

Prototype MVP · authentification, téléversement, liens de partage temporaires

Soutenance Projet 4 — Parcours Architecte logiciel

Ethan Legros — Référent technique senior, DataShare

Septembre 2026

Contexte et mission

Ce que DataShare attendait du prototype



Le produit

DataShare, jeune entreprise numérique. Cible : freelances et petites entreprises qui envoient des fichiers à des clients.



Le besoin

Transférer un fichier via un lien de téléchargement temporaire — sans imposer de compte au destinataire.



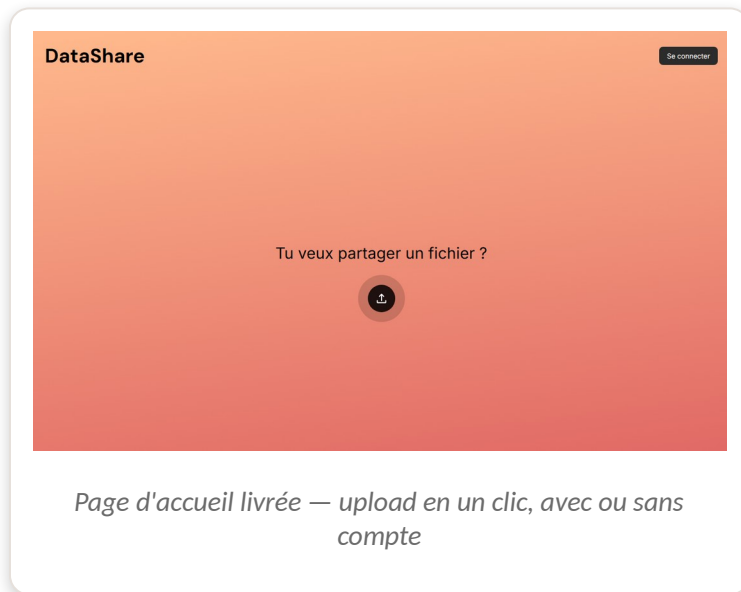
La commande

Un MVP fonctionnel en 4 semaines, présenté à des investisseurs, conforme aux maquettes Figma de l'UX designer.



Mon rôle

Référent technique senior : architecture, modèle de données, développement, qualité, supervision d'un copilote IA.



Page d'accueil livrée — upload en un clic, avec ou sans compte

Contraintes : stack à choisir parmi les options imposées • maquettes Figma à respecter • IA générative autorisée sur une seule user story, tracée dans Git

Périmètre fonctionnel : 10 user stories livrées

6 user stories du MVP obligatoire + les 4 fonctionnalités avancées optionnelles

MVP obligatoire



US01 Téléverser un fichier (compte), 1 Go max



US02 Télécharger via un lien unique, sans compte



US03 Créer un compte



US04 Se connecter — JWT



US05 Historique de mes fichiers : actifs / expirés



US06 Supprimer un fichier → lien invalidé

Fonctionnalités avancées (optionnelles) — toutes livrées



US07 Upload anonyme, sans compte



US08 Tags à l'upload + filtre — copilote IA supervisé



US09 Mot de passe sur le fichier (haché)



US10 Expiration 1-7 jours, purge automatique

Écrans conformes aux maquettes Figma (accueil, connexion, inscription, Mon espace, téléchargement) • responsive mobile

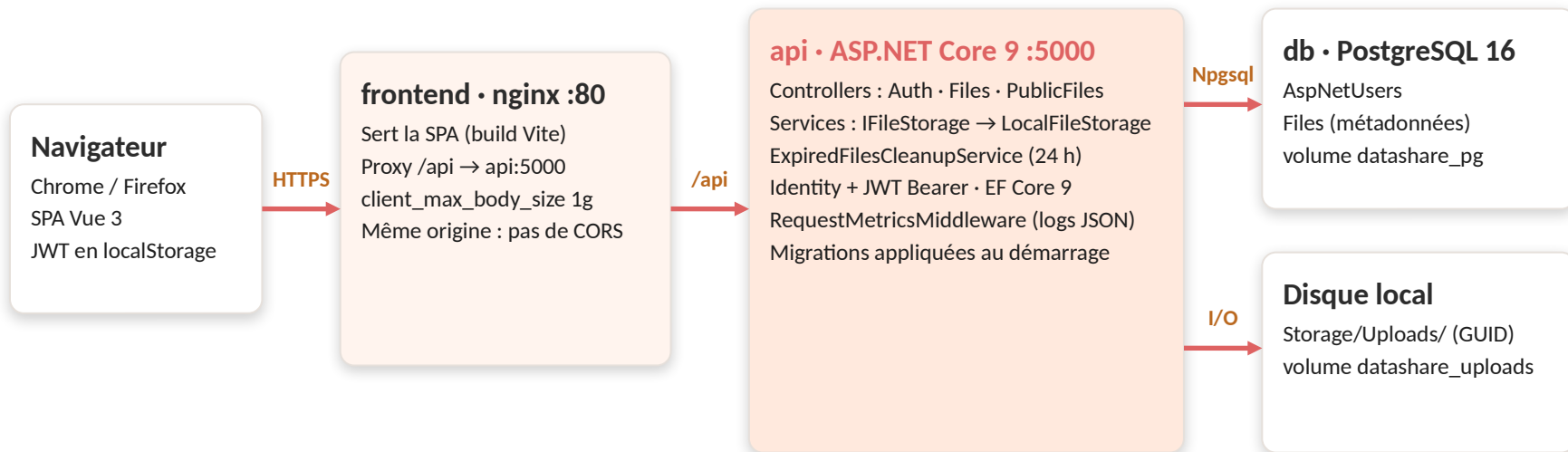
Choix techniques et justifications

Choisis parmi les options imposées par les spécifications — critère : livrer un MVP robuste en 4 semaines

Couche	Choix	Écarté	Pourquoi
Backend	ASP.NET Core 9 (C#)	Spring, NestJS, Symfony	Identity intégré (hachage PBKDF2, comptes sans code maison), EF Core (requêtes paramétrées), BackgroundService pour la purge, I/O asynchrones — P95 upload 125 ms sous 20 utilisateurs
Frontend	Vue 3 + TypeScript + Vite	Angular, React	Typage de bout en bout (vue-tsc), bundle initial 57 Ko gzip, lazy loading des vues, écosystème simple (Pinia, Vue Router, Axios)
Base de données	PostgreSQL 16	MongoDB	Données relationnelles (utilisateur → fichiers), type natif text[] pour les tags, transactions ACID, migrations EF Core
Stockage	Disque local derrière IFileStorage	AWS S3	Suffisant pour le MVP ; l'interface (Save / Delete / OpenRead) permet de passer à S3 sans toucher aux contrôleurs
Infra & tests	Docker Compose • xUnit • Cypress • k6	—	Démo reproductible en une commande (nginx sert la SPA et proxifie /api) ; pyramide de tests + test de charge

Architecture de la solution

Trois conteneurs Docker Compose • API REST stateless • fichiers hors base de données

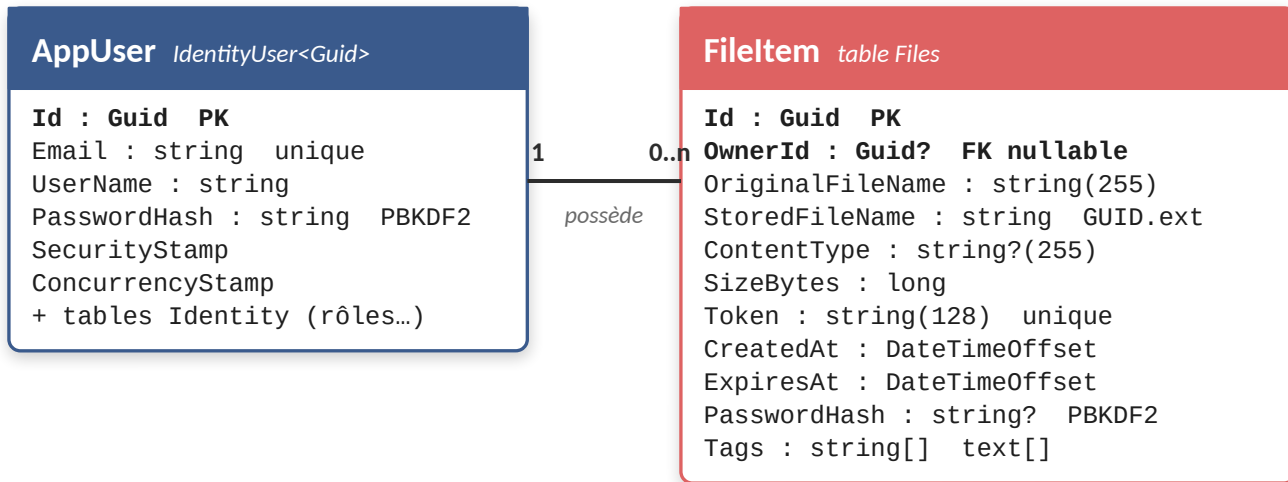


Pipeline HTTP RequestMetrics → Routing → CORS (origines autorisées) → Authentication (JWT) → Authorization ([Authorize]) → Controllers

Stateless côté API (aucune session serveur) → scalable horizontalement • seules les métadonnées vont en base, le contenu reste sur disque

Modèle de données

Deux entités — ASP.NET Core Identity pour les comptes, FileItem pour les fichiers (EF Core 9, PostgreSQL 16)



OwnerId nullable

L'upload anonyme (US07) crée un fichier sans propriétaire : il n'apparaît dans aucun historique.



Token ≠ Id

Le lien public utilise un token aléatoire de 32 octets (CSPRNG, Base64Url, index unique), jamais l'identifiant technique.



Contenu hors base

Seules les métadonnées sont en PostgreSQL ; le fichier vit sur disque sous un nom GUID (anti path traversal).



Tags en text[]

Type natif PostgreSQL, dédoublés sans casse, 20 max — pas de table de jointure pour un MVP.

4 migrations EF Core (InitialAuth → AddFiles → OwnerId nullable) appliquées automatiquement au démarrage de l'API — aucun script SQL manuel

Contrat d'interface — API REST

docs/openapi.yaml (OpenAPI 3, validé) · Swagger UI en développement · API_REFERENCE.md

Méthode	Route	Rôle	Accès	Réponses
POST	/api/auth/register	Créer un compte (US03) → JWT	Public	200 · 400 (email pris, mdp < 8)
POST	/api/auth/login	Se connecter (US04) → JWT 8 h	Public	200 · 401 (email inconnu = mdp faux)
POST	/api/files	Upload multipart : file, expiresInDays, password, tags (US01/08/09/10)	JWT	201 · 400 (.exe, > 1 Go, durée) · 401
GET	/api/files/me?status=	Historique all active expired (US05)	JWT	200 · 401
GET	/api/files/{id}	Métadonnées d'un de mes fichiers	JWT	200 · 404 (jamais 403)
DELETE	/api/files/{id}	Supprimer disque + base (US06)	JWT	204 · 404
POST	/api/public/files	Upload anonyme (US07)	Public	201 · 400 · 403 si connecté
GET	/api/public/files/{token}	Métadonnées via le lien (US02)	Public	200 · 404 · 410 expiré
POST	/api/public/files/{token}/download	Télécharger, mot de passe dans le corps (US02/09)	Public	200 flux · 401 mdp · 404 · 410
GET	/api/health	Santé (healthcheck Docker, scripts)	Public	200

Convention : identité toujours extraite du JWT côté serveur · erreurs métier explicites (400 texte, 401/404/410 JSON) · réponses JSON camelCase

Sécurité et gestion des accès

Défense en profondeur — chaque mécanisme est dans le code, référencé dans SECURITY.md et couvert par des tests



Authentification

- JWT HS256 signé, 8 h, validé à chaque requête
- Mots de passe hachés PBKDF2 (Identity)
- Clé ≥ 32 caractères vérifiée au démarrage



Isolation des données

- Identité extraite du token, jamais du client
- Filtre OwnerId sur chaque requête privée
- 404 sans fuite · tests d'isolation croisée



Liens de partage

- Token CSPRNG 32 octets $\neq$ identifiant
- Mot de passe fichier haché, envoyé en POST
- Lien expiré $\rightarrow$ 410, supprimé $\rightarrow$ 404



Fichiers

- Renommés GUID : anti path traversal
- Exécutables refusés (.exe .bat .msi .ps1...)
- 1 Go max · purge quotidienne



Validation double

- Client pour l'UX, serveur comme autorité
- EF Core : requêtes paramétrées, zéro SQL brut
- CORS limité aux origines configurées



Secrets et logs

- Aucun secret dans Git : .env / user-secrets
- Logs structurés sans mot de passe ni token
- Audit dépendances : 0 vulnérabilité (13/09)

Roadmap sécurité (hors MVP) : HTTPS/HSTS sur le reverse proxy · rate limiting sur login et téléchargement · refresh tokens · scan antivirus des uploads

Démonstration

Parcours complet en direct sur <http://localhost> — fonctionnalités clés, expérience utilisateur, gestion des erreurs

1

Créer un compte, se connecter

JWT côté client ; erreurs claires : email pris, mot de passe court, identifiants faux

2

Téléverser un fichier

Expiration 3 jours, mot de passe, tags — lien de partage copié en un clic

3

Mon espace

Historique, onglets actifs / expirés, filtre par tag, accès au lien

4

Côté destinataire (fenêtre privée)

Lien → nom et taille → mot de passe → téléchargement, nom d'origine conservé

5

Gestion des erreurs

Mauvais mot de passe · fichier .exe refusé · lien inconnu → messages explicites, jamais d'écran blanc

6

Supprimer, puis observer

Le lien devient invalide ; docker compose logs api → métriques structurées de chaque requête

DataShare

Se connecter

Connexion

Email

Saisissez votre email...

Mot de passe

Saisissez votre mot de passe...

Créer un compte

Connexion

Écran de connexion (maquette Figma respectée)

Plan B : captures d'écran des 5 pages et de Swagger dans docs/soutenance/screenshots

Qualité : tests et couverture

Pyramide de tests exécutable — plan de tests par user story avec critères d'acceptation (TESTING.md)

41

tests xUnit backend
2 unitaires · 39 d'intégration

83 %

couverture du code métier
(cible ≥ 70 %, hors migrations)

5

scénarios Cypress end-to-end
contre la stack Docker

0

erreur ESLint · type-check strict
nullable C# activé



Unitaires

- ExpiredFilesCleanupService : purge disque + base
- Moq : le stockage en échec est logué, jamais fatal



Intégration HTTP

- WebApplicationFactory + EF Core InMemory
- TestAuthHandler multi-utilisateurs (X-Test-UserId)
- Tous les endpoints, codes d'erreur, isolation, 410



End-to-end

- Inscription → upload → lien → téléchargement tiers
- Tags, mot de passe, limites (vide, 5 Mo, > 1 Go)
- Cypress sur http://localhost (nginx + API + BDD)

Exemples de critères : fichier d'un autre utilisateur → 404 et fichier intact · lien expiré → 410 sur métadonnées et téléchargement · mauvais mot de passe → 401 · .exe → 400 « Forbidden file type »

Performance et observabilité

Test de charge k6 · budget front Lighthouse · logs structurés exploitables (PERF.md)

Backend — k6, 20 utilisateurs × 30 s, upload 100 Ko authentifié

125 ms

P95 upload (cible < 500)

0 %

erreurs sur 500 uploads

32/s

requêtes par seconde

Frontend — build de production

57 Ko

JS initial gzip (budget < 100)

96

Lighthouse Performance

76

Accessibility (contrastes charte)

Pistes : streaming des gros fichiers sans tampon, compression des réponses, pagination de /files/me, polices auto-hébergées (FCP), stockage S3 + CDN

Logs structurés (JSON, une ligne par événement)

```
"Category": "...RequestMetricsMiddleware"  
"Method": "POST", "Path": "/api/files"  
"StatusCode": 201, "ElapsedMs": 96.4  
"RequestBytes": 205112
```

```
"Message": "File uploaded 3fa8... (204800 bytes,  
application/pdf) by user 9b1d..., expires ...,  
passwordProtected=True, tags=2"
```

- Chaque requête : méthode, route, statut, durée, octets
- Upload / téléchargement / suppression : id, taille, type
- Exploitables par grep (P95, volume) ou Seq / Loki / ELK

Documentation et maintenance

Tout est dans le dépôt GitHub — prise en main en une commande, procédures écrites pour la suite



README

prérequis, docker compose up en 1 commande, dev local, utilisation



Documentation technique (PDF, 29 p.)

architecture, choix, modèle de données, API, sécurité, qualité, installation, IA



OpenAPI + API_REFERENCE.md

contrat machine validé + référence lisible, Swagger en dev



TESTING • SECURITY • PERF • MAINTENANCE

plan de tests, audits datés et décisions, k6/Lighthouse/logs, exploitation



scripts/

deploy.ps1 / .sh, db-backup, db-restore, verify.ps1 (tests + couverture + audits)



Historique Git

conventional commits, branches main / develop, contributions IA isolées

Maintenance (MAINTENANCE.md)

- Mises à jour : audit npm + NuGet à chaque livraison, patch/mineures mensuelles, majeures 1-2×/an sur branche dédiée
- Risques → parades : jamais npm audit fix --force ; pg_dump avant changement de majeure PostgreSQL ; dotnet test + build front après chaque mise à jour
- Bug : reproduire (test qui échoue) → isoler la couche → branche fix/ → tests → revue → merge
- Sauvegardes : base (pg_dump) + volume des fichiers, ensemble ; migrations EF appliquées au démarrage
- Cas réel : CVE Microsoft.OpenApi (08/2026) — risque évalué, patch mineur, tests verts ; 13 avis npm (09/2026) corrigés

Pilotage du copilote IA — US08 « Tags »

Une user story confiée à l'IA comme à un développeur junior : cadrage, relecture, correctifs, traçabilité (AI_USAGE.md)



1 • Tâche assignée

Prompt tracé dans IA_Instructions.txt — front uniquement :

« Chips de tags à l'upload (Enter), affichage dans Mon espace, filtre local case-insensitive.
Contraintes : ne pas modifier le back, aucune nouvelle lib, style datashare.css, trim / pas de doublon / 24 caractères max. »

- Livré en ~15 min pour ~200 lignes (≈ 2 h à la main)
- Fichiers : UploadView, MeView, datashare.css — back intact



2 • Supervision et correctifs

- Relecture complète du diff : back intact, pas de dépendance ajoutée, styles cohérents
- Tests manuels des cas limites : tag vide, > 24 caractères, doublon exact et doublon de casse
- Manques trouvés : aucun message d'erreur utilisateur, validation UI incomplète
- Correctifs en commit séparé (revue humaine) ; le back reste l'autorité (déduplication, 20 max)



3 • Traçabilité et bilan

56f411d feat(ai): add tags UI and filtering
a8c23fc fix(tags): show clear UI errors...

- Apports : gain de temps, code dans le style existant, bon en audit et recherche technique
- Limites : UX d'erreur oubliée, aucun test proposé, cas limites non traités
- Conclusion : l'IA accélère, la revue humaine décide — même posture pour l'audit, le debug et la doc

Difficultés rencontrées et solutions

Ce qui n'a pas marché du premier coup — et ce que j'en retiens

Difficulté	Solution apportée	Leçon
Upload anonyme (US07) : le modèle exigeait un propriétaire pour chaque fichier	OwnerId rendu nullable via deux migrations EF Core ; endpoint public dédié, refusé aux utilisateurs connectés (403)	Concevoir le modèle avec les US avancées en tête dès le MCD
Vulnérabilité CVE sur Microsoft.OpenApi et 16 avis npm en août	Analyse du risque réel, suppression d'une dépendance morte (react-router-dom), mise à jour ciblée, tests verts, décisions documentées	Auditer à chaque livraison ; moins de dépendances = moins de surface
Une mise à jour de sécurité d'axios a cassé le type-check	Détecté par npm run build, corrigé en une ligne	Le typage strict est un filet de sécurité, pas une contrainte
Tester l'isolation entre utilisateurs avec un seul faux utilisateur de test	TestAuthHandler enrichi : en-tête X-Test-UserId pour incarner un second compte → 7 tests de sécurité réels	L'infrastructure de test doit permettre les scénarios de sécurité
Accessibility Lighthouse à 76 : contrastes issus de la charte Figma	Corrections hors charte (lang, titre, meta), point remonté à l'UX designer plutôt que modifié unilatéralement	Respecter la maquette n'exclut pas de la challenger

DataShare

Livré en 4 semaines

- 10 / 10 user stories, écrans conformes aux maquettes
- Application conteneurisée, lancée en une commande
- 41 tests, 83 % de couverture, 5 scénarios E2E, 0 vulnérabilité
- Sécurité et isolation vérifiées par des tests
- Documentation complète et alignée sur le code

Prochaines itérations

- HTTPS sur le reverse proxy, rate limiting, refresh tokens
- Stockage S3 derrière IFileStorage, CDN pour les téléchargements
- Accessibilité : atelier contrastes avec l'UX designer
- Scan antivirus des uploads, streaming des gros fichiers
- Collecteur de logs (Seq / Loki) et tableau de bord des métriques

Merci — place aux questions

github.com/GadZouil/OC-P4-DataShare · [docs/documentation_technique.pdf](#)